Kombinatorikai algoritmusok
Kimerítő keresés

Szerző: Horváth Gyula

2025. november 18.

1. Bevezetés

A ***kimerítő keresés*** az egyik legáltalánosabb probléma-megoldási stratégia. A stratégia lényege, hogy sorba-veszi az összes lehetséges esetet, amely megoldás lehet és ellenőrzi, hogy teljesül-e az esetre a megoldás feltétele. Ha csak egy megoldást keresünk, akkor az első megtalált esetnél befejeződik a keresés. Ezzel a stratégiával Megadhatjuk a probléma összes megoldását és egy optimális megoldást is. Optimális megoldás esetén olyan megoldást keresünk, amelyre adott célfüggvény értéke minimális/maximális.
Adott probléma összes lehetséges megoldása nagyon gyakran kifejezhető valamilyen kombinatorikus objektummal. A legfontosabbak: **permutációk**, **részhalmazok**, **kombinációk**, **variációk**.
A továbbiakban ilyen kombinatorikus objektumokat előállító algoritmusokat tárgyalunk.
Az általánosság megszorítása nélkül feltehetjük, hogy a probléma megoldásnak kifejezése az adatok indexeinek megadásával történik, ezért a továbbiakban a kombinatorikus objektumok alaphalmaza az `{1, ..., n}` vagy a `{0, ..., n−1}` halmaz lesz.

Megjegyezzük, hogy a kimerítő keresés futási ideje általában exponenciálisan nő a feladat méretének függvényében, ezért csak korlátozottan alkalmazható.

2. Permutációk

Adott `n` pozitív egész számra állítsuk elő az `{1, ..., n}` halmaz összes permutációját.
Számos algoritmus ismert erre a feladatra.

2.1. Könyvtári megoldás

A **C++** standard fejlesztő környezetben van beépített megoldás, a next_permutation függvény.
A permutálandó halmaz elemeinek kezdeti permutációját egy tömbben, vagy dinamikus tömbben (vector) kell megadni. Pontosabban, iterátor paraméterrel kell megadni az első és az utolsó utáni pozíciót. A pozíciók tetszőlegesek lehetnek, tehát a kezdeti permutáció akármilyen részsorozat lehet. A függvény mindaddig igaz értéket ad, ameddig a az aktuális permutáció nem egyezik meg a kezdetivel. A két iterátor által meghatározott intervallumon kívüli elemek változatlanok maradnak.
A C++20 szabvány szerint a permutációk lexikografikus sorrendben lesznek felsorolva.

```
1 vector<int> P = {1,2,3,4,5};
2 do{//a P permutáció feldolgozása
3   for (int x : P)cout << x << " "; cout<<endl;
4 }while (next_permutation(P.begin()+1,P.begin()+4));
```

A kimenet:

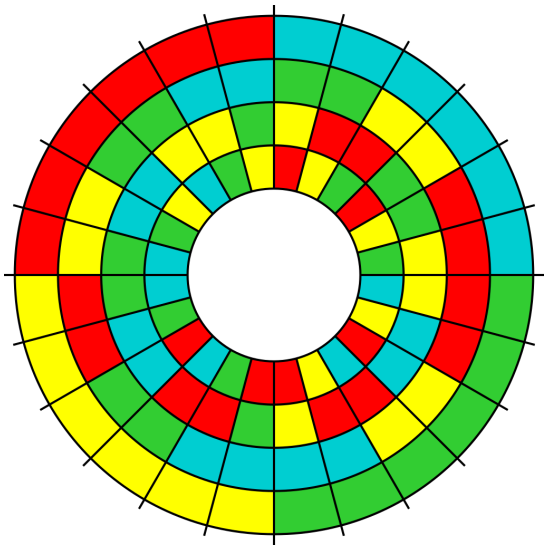
```
1 2 3 4 5
1 2 4 3 5
1 3 2 4 5
1 3 4 2 5
1 4 2 3 5
1 4 3 2 5
```

2.1.1. B. R. Heap algoritmusa

```
void HeapP(vector<int>& P, int n){
  //B.R. Heap algoritmusa
  if(n==1){//feldolgozás
    for(int x:P) cout<<x<<" "; cout<<endl;
    return;
  }
  for(int i=0;i<n;i++){
    HeapP(P, n-1);
    if(n%2==0)
      swap(P[i], P[n-1]);
    else
      swap(P[0], P[n-1]);
  }
}
```

Permutáció rozetta reprezentálása.

```
1 2 3 4
2 1 3 4
3 1 2 4
1 3 2 4
2 3 1 4
3 2 1 4
4 2 1 3
2 4 1 3
1 4 2 3
4 1 2 3
2 1 4 3
1 2 4 3
1 3 4 2
3 1 4 2
4 1 3 2
1 4 3 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
2 4 3 1
4 2 3 1
3 2 4 1
2 3 4 1
```



2.1.2. Hatékony iteratív előállítás

```
void IPermutaciok(int n){
//Összes permutáció iteratív előállítása
//B.R.Heap iteratív algoritmusa
int C[n];
int P[n];
    for(int i=0;i<n;i++) {C[i]=0; P[i]=i+1;}
    for(int k=0;k<n;k++) cout<<P[k]<<" "; cout<<endl;
    int i=1;
    while(i<n){
        if(C[i]<i){
            if(i%2==0){
                swap(P[0], P[i]);
            }else{
                swap(P[C[i]], P[i]);
            }
            for(int k=0;k<n;k++) cout<<P[k]<<" "; cout<<endl;
            C[i]++;
            i=1;
        }else{
            C[i]=0;     i++;
        }
    }
}
```

A későbbiekben szükségünk lesz olyan algoritmusra, amely nem csak az összes permutációt állítja elő, hanem azok kezdőszeleteit is. A következő rekurzív algoritmus egy lehetséges megoldást ad.

```
void Permutaciok(vector<int>& P, int n, int i){
  //permutációk és kezdőszeketek előállítása
  //A P[0],...,P[i-1] kezdőszelet feldolgozása
  for(int j=0;j<i;j++) cout<<P[j]<<","; cout<<endl;

  for(int k=i;k<n;k++){//P[i] lehet P[i..n-1] mindegyike
    swap(P[i], P[k]);
    Permutaciok(P, n, i+1);
    swap(P[i], P[k]);
  }
}
```

2.2. Lexikografikus sorrend szerinti követő permutáció

`P = (4, 3, 2, 6, 8, 7, 5, 1)` követője:
`Q = (4, 3, 2, 7, 1, 5, 6, 8)` Legyen `i` a legnagyobb olyan index, hogy van olyan `i < k`, hogy `P[i] < P[k]`
Ha több ilyen is van, akkor az, amelyre `P[k]` a legkisebb.
Ekkor `P[i] < P[i + 1] > P[i + 2] > ··· > P[k] > ··· > P[n]`
Cseréljük meg `P[i]`-t és `P[k]`, majd rakjuk növekvő sorrendbe `P[i+1], ..., P]n]` elemeket. Mivel ez a sorozat csökkenő, ezért elempárok cseréjével sorba lehet rakni.

```
1  void Kovet(int P[], int n){
2      int i=n-1;
3      while(i>=1 && P[i]>P[i+1]) i--;
4      if(i==0) return; //P csökkenő, nincs rákövetkezője
5  //P[i]<P[i+1]>P]i+2]>...>P[n]
6  //i<k<=n:P[i]<P[k]
7      int k=n;
8      while(P[k]<P[i]) k--;
9      swap(P[i], P[k]);
10     i++; k=n;
11     while(i<k){
12         swap(P[i], P[k]);
13         i++; k--;
14     }
15 }
```

2.3. Lexikografikus sorrend szerinti előző permutáció

Van beépített megvalósítása: prev_permutation.
`P = (4, 3, 2, 7, 1, 5, 6, 8)` előzője:
`Q = (4, 3, 2, 6, 8, 7, 5, 1)` Legyen `i` a legnagyobb olyan index, hogy van iolyan `i < k`, hogy `P[i] > P[k]`
Ha több ilyen is van, akkor az, amelyre `P[k]` a legkisebb.
Ekkor `P[i] > P[i + 1] < P[i + 2] < ··· < P[k] < ··· < P[n]`
Cseréljük meg `P[i]`-t és `P[k]`, majd rakjuk növekvő sorrendbe `P[i+1], ..., P]n]` elemeket. Mivel ez a sorozat növekvő, ezért elempárok cseréjével sorba lehet rakni.

```
1  void Elozo(int P[], int n){
2      int i=n-1;
3      while(i>=1 && P[i]<P[i+1]) i--;
4      if(i==0) return;
5  //P[i]>P[i+1]<P]i+2]<...<P[n]
6  //i<k<=n:P[i]>P[k]
7      int k=n;
8      while(P[k]>P[i]) k--;
9      swap(P[i], P[k]);
10     i++; k=n;
11     while(i<k){
12         swap(P[i], P[k]);
13         i++; k--;
14     }
15 }
```

2.4. Lexikografikus sorrend szerint i-edik permutáció

Technikai okok miatt az $\{0, 1, \ldots, n-1\}$ halmaz permutációit vesszük.
Nyilvánvaló, hogy n elem összes permutációját tekintve $(n-1)!$ olyan van, amelyik 0-el, 0-vel, ..., $n-1$-el kezdődik.
A keresett, tehát az i-edik permutáció a $P = (p_1, \ldots, p_n)$ sorozat.
Legyen $index = (0, 1, \ldots, n-1)$ sorozat.
$p_1 = i/((n-1)!)$
Vegyük az összes olyan permutációt lexikografikus sorrendben, amelyek a $index$ sorozatból p_1-edik kihagyásával kaphatók.
Akkor ebben a sorrendben az $i - (p_1 * (n-1)!)$-adik permutációt kell keresni (az első tagját). És így tovább.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  void i_Permutacio(const int n, long long i){
4      int j,n1=n;
5      long long fakt=1;
6      for(int k=2;k<=n;k++) fakt*=k;
7      if(i>=fakt) {cout<<i<<">="<<fakt<<endl; return;}
8      vector<int> index;
9      for(int k=0;k<n; k++) index.push_back(k);
10     int perm[n];
11
12     for (int k=0; k<n; k++, n1--) {
13         fakt /= n1;
14         j = (i / fakt);
15         perm[k] = index[j];
16         i -= (fakt * j);
17         index.erase(index.begin()+j);
18     }
19     for(int k=0;k<n;k++) cout<<perm[k]+1<<' '; cout<<endl;
20 }
21 int main(){
22     i_Permutacio(10, 1000000);
23 //3 8 9 4 10 2 6 7 1 5
24 }
```

2.5. Feladat: Ültetés

Programozási csapatverseny eredményhirdetését úgy szervezik, hogy minden csapat egy külön kerek alakú asztalhoz lesz leültetve. Minden csapattól megkérdezték, hogy milyen sorrendben akarnak ülni. Pontosabban minden csapattagnak meg kellett adni két másik csapattagot, hogy úgy szeretne ülni, hogy legalább az egyik megadott csapattag mellette üljön valamelyik oldalán.
Készíts programot, amely a kívánságok alapján megad egy olyan ültetési sorrendet, amely teljesíti minden tag kívánságát!

Bemenet

A standard bemenet első sorában egy egész szám van, a csapattagok N száma. A további N sor mindegyike két egész számot tartalmaz a_i, b_i, ami azt jelenti, az i-edik csapattag azt kéri, hogy mellette az a_i m vagy a b_i csapattag üljön. A csapattagokat az $1, \ldots, N$ számokkal azonosítjuk.

Kimenet

A standard kimenet első és egyetlen sorába N számot kell írni, a csapat egy megfelelő ültetési sorrendjét! Ha nincs megoldás, akkor egy üres sort kell kiírni. Több megoldás esetén bármelyik megadható.

Példa

Bemenet

```
8
4 8
4 3
6 2
1 3
8 2
3 7
8 1
5 7
```

Kimenet

```
1 8 5 2 4 3 6 7
```

Korlátok

$1 < N \leq 12$
$a_i \neq b_i$ és $1 \leq a_i \leq N$ és $1 \leq b_i \leq N$
Időlimit: 1.0 másodperc
Memórialimit: 64 MB.

2.6. Megoldás

Jelölje `P` a lehetséges megoldás permutációt, `F` pedig a kívánságok listáját, tehát az `i`-edik tag az `F[i].first` vagy az `F[i].second` tag mellett szeretne ülni. Ekkor a a megoldás feltétele az alábbi algoritmussal számítható ki.

```
1  bool Megoldas(vector<int>& P, vector<pair<int,int>>& F){
2    bool van=true;
3    int n=P.size();
4    int kovet,eloz;
5    for(int i=0;i<n;i++){
6      kovet=P[(i+1)%n]; eloz= i>0 ? i-1: n-1;
7      if( (F[P[i]].first!=kovet && F[P[i]].second!=kovet) &&
8      (F[P[i]].first!=eloz && F[P[i]].second!=eloz) ){
9        van=false; break;
10     }
11   }
12   return van;
13 }
```

Kimerítő kereséssel a feladat megoldása:

```
1  int main(){
2    int n,x,y;
3    vector<pair<int,int>> F;
4    F.push_back({0,0});
5    vector<int> P;
6    cin>>n;
7    for(int i=1;i<=n;i++){
8      cin>>x>>y;
9      F.push_back({x,y});
10     P.push_back(i);
11   }
12   bool van=false;
13   do{
14     if(Megoldas(P, F)) {van=true; break;}
15   }while (next_permutation(P.begin(), P.end()));
16
17   if(van) for(int x:P) cout<<x<<" "; cout<<endl;
18 }
```

3. Variációk

3.1. definíció. n *elemű halmazból hányféleképpen lehet kiválasztani* $1 \leq k \leq n$ *különböző elemet, ha a kiválasztás sorrendje is számít?*
Ezt az értéket n *elem* k*-ad osztályú ismétlés nélküli variációjának nevezzük. Az* (n, k) *elnevezést is használjuk erre.*
Értéke $n \cdot (n-1) \cdots (n-k+1) = \frac{n!}{(n-k)!}$

3.1. Az összes (n, k) variáció előállítása.

```
1  void Variaciok(vector<int> P, int n, int k, int i){
2    //0<=i<=k, előállítja az összes olyan variációt,
3    //amelynek P[0],..,P[i-1] a kezdőszelete
4    if(i==k){//P feldolgozása
5      for(int kk=0;kk<k;kk++) cout<<P[kk]<<' '; cout<<endl;
6      return;
7    }
8    for(int ii=i;ii<n;ii++){//P[i] lehet P[i..n-1] mindegyike
9      swap(P[i], P[ii]);
10     Variaciok(P,n,k,i+1);
11     swap(P[i], P[ii]);
12   }
13 }
```

A `(4, 3)` variációk a fenti algoritmus előállításával.

```
1 2 3
1 2 4
1 3 2
1 3 4
1 4 3
1 4 2
2 1 3
2 1 4
2 3 1
2 3 4
2 4 3
2 4 1
3 2 1
3 2 4
3 1 2
3 1 4
3 4 1
3 4 2
4 2 3
4 2 1
4 3 2
4 3 1
4 1 3
4 1 2
```

4. Feladat: Hozzárendelési feladat.

Egy építkezésen `M` munkát kell elvégezni. `V` számú alvállalkozó adott be pályázatot. Mindegyik megadta, hogy mely munkákat tudja elvégezni. Minden alvállalkozó csak egy munkára kaphat megbízást.
Készíts programot, amely minden munkára kiválaszt egy megfelelő pályázót! Jelezni kell, ha nincs megoldás !

Bemenet

A standard bemenet első sorában van a munkák `M` száma és a pályázó alvállalkozók `V` száma. A további `V` sor közül az `i`-edik sor azokat a munkákat tartalmazza, amelyeket az `i`-edik pályázó el tud végezni. A munkákat az `1, ..., M` számokkal, a pályázó alvállalkozókat pedig az `1, ..., V` számokkal azonosítjuk.

Kimenet

A standard kimenet első és egyetlen sorába `M` számot kell írni, az i-edik szám annak a pályázónak a sorszáma legyen, amelyik az `i`-eik munkát fogja elvégezni! Ha nincs megoldás, akkor egy üres sort kell kiírni.

Példa

Bemenet

```
4 6
2 3 1
3 1
1 3
1 2
2 3
2 1 4
```

Kimenet

```
1 4 3 6
```

Korlátok

$1 < M \leq 16, M \leq V \leq 16$
Időlimit: `1.0` másodperc
Memórialimit: `64` MB.

4.1. Megoldás

Jelölje `VM[v]` azon munkák halmazát, amelyeket az `v` alvállalkozó el tud végezni.
A feladat megoldása megadható egy olyan $f : \{1, \ldots, M\} \to \{1, \ldots, V\}$ függvénnyel, amelyre teljesül az alábbi két feltétel:

$$f(i) \neq f(j) \text{ minden } 1 \leq i < j \leq M \text{ esetén}$$

$$m \in VM[f(m)] \text{ minden } 1 \leq m \leq M \text{ esetén}$$

Tehát a megoldás megadható az `(f(1), ..., f(M))` sorozattal, ami nem más, mint `V`-nek `M`-ad osztályú variációja.
Tehát minden lehetséges megoldása a feladatnak megadható egy-egy variációval.
Alkalmazzuk a kimerítő keresés stratégiáját.
Az 1. feltétel nyilvánvalóan teljesül, mivel a variáció elemei különbözők. A második feltétel ellenőrzéséhez az `E[i]` halmazokat ábrázoljuk `vector < set < int >> VM` dinamikus tömbbel.

```
1   vector<set<int>> VM;
2   vector<int> f;
3 void Megoldas(vector<int> P, vector<set<int>> VM, int k){
4   bool jo=true;
5   for(int i=0;i<k;i++){
6     if(VM[P[i]].find(i+1)==VM[P[i]].end()){
7       jo=false; break;
8     }
9   }
10  if(jo)//megoldást találtunk, feljegyezzük
11  for(int i=0;i<k;i++) f.push_back(P[i]);
12 }
```

```
1 void Variaciok(vector<int> P, int n, int k, int i){
2   if(i==k){//feldolgozás
3     Megoldas(P,VM,k);
4     return;
5   }
6   for(int ii=i;ii<n;ii++){//P[i] lehet P[i..n-1] mindegyike
7     swap(P[i], P[ii]);
8     Variaciok(P,n,k,i+1);
9     if(f.size()>0) return;
10    swap(P[i], P[ii]);
11  }
12 }
```

```
int main(){
  int M,V,x;
  string s;
  cin>>M>>V;
  VM=vector<set<int>>(V+1);
  getline(cin, s);
  for(int v=1;v<=V;v++){
    while(cin.peek()!='\n'){
      cin>>x;
      VM[v].insert(x);
    }
    getline(cin, s);
  }
  vector<int> P;
  for(int i=1;i<=V;i++) P.push_back(i);
  Variaciok(P, V, M, 0);
  for(int x:f) cout<<x<<" "; cout<<endl;
}
```

5. Kombinációk

5.1. definíció. *n elemű halmazból hányféleképpen lehet kiválasztani $k \leq n$ különböző elemet, ha a kiválasztás sorrendje nem számít?*
Ezt az értéket n elem k-ad osztályú ismétlés nélküli kombinációjának nevezzük. Az (n, k) kombinációk elnevezést is használjuk.

$$C_n^k = \binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

$$\binom{n}{k} = \frac{n \cdot (n-1) \cdots (n-k+1)}{k!}$$

(Binomiális együtthatók)

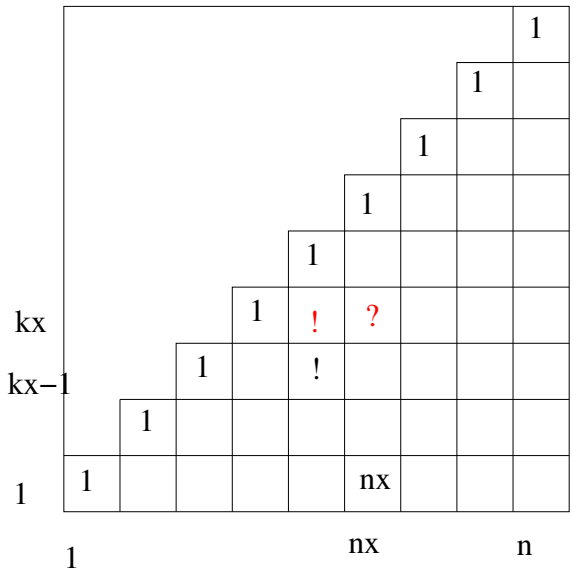
5.1. Hatékony kiszámítás, dinamikus programozás módszerével

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

$$C(n, 1) = n$$

$$C(n, n) = 1$$

$$C(n, k) = C(n-1, k-1) + C(n-1, k) \text{ ha } 1 < k < n$$



1. ábra. $C(n, k) = C(n-1, k-1) + C(n-1, k)$, kitöltési sorrend: oszloponként fölülről lefelé.

									1
								1	9
							1	8	36
						1	7	28	84
					1	6	21	56	126
				1	5	15	35	70	126
			1	4	10	20	35	56	84
		1	3	6	10	15	21	28	36
	1	2	3	4	5	6	7	8	9

1

1 n

2. ábra

```
long long C(int n, int k){
    long long Oszlop[n+1];
    Oszlop[1]=1;
    for(int nx=2;nx<=n;nx++){//nx. oszlop kitöltése
        Oszlop[nx]=1;
        for(int kx=nx-1;kx>1;kx--)
            Oszlop[kx]+=Oszlop[kx-1];
        Oszlop[1]=nx;
    }
    return Oszlop[k];
}
```

5.2. Az összes (n, k) kombináció előállítása.

```
1  void Kombinaciok(int n, int k, int i, vector<int>& R){
2    //Összes (n,k) kombináció előállítása, amelynek
3    //kezdőszelete R[0],...,R[R.size()-1]
4    //az első i elemből
5    if(k==R.size()){//R feldolgozása
6      for(int x:R) cout<<x<<" "; cout<<endl;
7      return;
8    }
9    for(int j=i;j<=n;j++){
10     R.push_back(j);
11     Kombinaciok(n, k, j+1, R);
12     R.pop_back();
13   }
14 }
```

Az összes (5,3) kombináció

```
1,2,3
1,2,4
1,2,5
1,3,4
1,3,5
1,4,5
2,3,4
2,3,5
2,4,5
3,4,5
```

6. Feladat: Koalíciók

Demokri országban többségi parlamenti rendszer működik. Ez azt jelenti, hogy csak olyan pártokat tartalmazó koalíció alakíthat kormányt, amelyek együttesen rendelkeznek a parlamenti helyek többségével. Ismerjük a legutóbbi parlamenti választás eredményét, tehát azt, hogy mely pártok hány parlament mandátumot szereztek.
Készíts programot, amely minden parlamenti pártra megadja, hogy hány különböző koalíciónak lehetne a tagja, amelynek pontosan `K` tagja van!

Bemenet

A standard bemenet első sorában van a parlamenti pártok `P` száma és a lehetséges koalíciós pártok `K` száma. A második sor pontosan `P` pozitív egész számot tartalmaz, az `i`-edik szám az `i`-edik parlamenti párt mandátumainak a száma. A parlamenti pártokat az `1, ..., P`. számokkal azonosítjuk.

Kimenet

A standard kimenet első és egyetlen sorába `P` számot kell írni, az i-edik szám azon többségi koalíciók száma legyen, emelyeknek az `i`-eik párt tagja lehet!

Példa

Bemenet	Kimenet
`6 3` `10 2 8 5 6 36`	`8 2 5 4 5 3`

Korlátok

$1 < P \leq 20$, $1 \leq p_i \leq 100\,000$
Időlimit: `1.0` másodperc
Memórialimit: `64` MB.

6.1. Megoldás

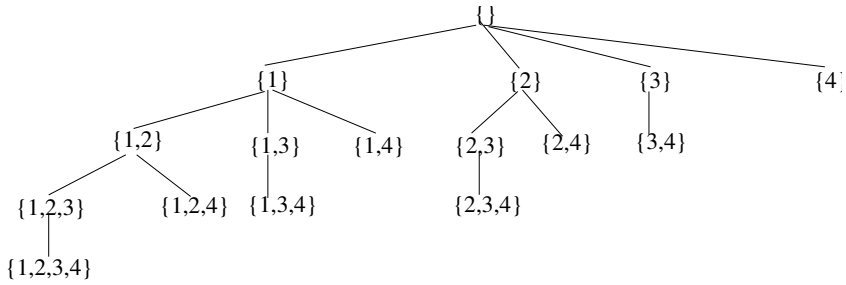
Vegyük sorra az összes (P,K) kombinációt. Ha a benne szereplő pártok mandátumainak összege nagyobb, mint a parlamenti helyek számának a fele, akkor a koalíció minden pártjának a megoldási értékét növeljük eggyel.

```
1  #include <bits/stdc++.h>
2  #define maxP 21
3  using namespace std;
4  int P[maxP];
5  int M[maxP];
6  int Ossz=0;
7
8  void Kombinaciok(int n, int k, int i, vector<int>& R){
9    //Összes (n,k) kombináció előállítása rekurzívan
10   //Megoldáskezdemény:R[0],...,R[R.size()-1]
11   //az első i elemből
12   if(k==R.size()){//R feldolgozása
13     int nr=0;
14     for(int x:R) nr+=P[x];
15     if(2*nr>Ossz){
16       for(int x:R) {
17         cout<<x<<":"<<P[x]<<"+";
18         M[x]++;
19       }cout<<endl;
20     }
21     return;
22   }
23   for(int j=i;j<=n;j++){
24     R.push_back(j);
25     Kombinaciok(n, k, j+1, R);
26     R.pop_back();
27   }
28 }
```

```
int main(){
  vector<int> R;
  int p,k;
  cin>>p>>k;

  for(int i=1;i<=p;i++){
    cin>>P[i];
    Ossz+=P[i];
  }
  Kombinaciok(p,k,1, R);
  for(int i=1;i<=p;i++) cout<<M[i]<<" "; cout<<endl;
}
```

7. Részhalmazok

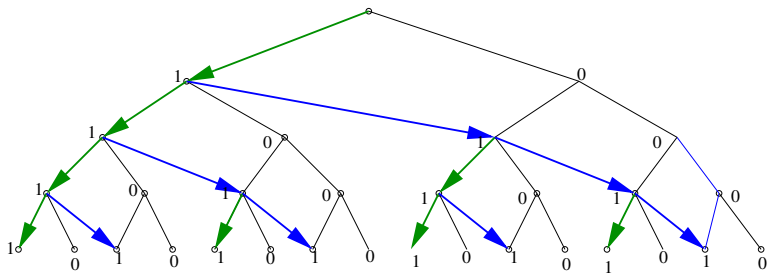


3. ábra. A részhalmazok fája

7.1. Összes részhalmaz rekurzív előállítása

```
void RReszhalmazok(int n,int i, vector<int>& R){
//Rekurzív megoldás
//Megoldáskezdemény:R[0],...,R[R.size()-1]
//az első i elemből
//Az R részhalmaz feldolgozása:
    for(int x:R) cout<<x<<","; cout<<endl;

    for(int j=i;j<=n;j++){
        R.push_back(j);
        RReszhalmazok(n, j+1, R);
        R.pop_back();
    }
}
```



4. ábra. Részhalmazok bitvektoros reprezentációja `n = 4` esetén

Minden részhalmazt olyan bitsorozat reprezentál, amelynek az utolsó tagja igaz értékű. Az `Elso(R)` és a `Testver(R)` műveletekkel generálhatjuk az összes részhalmazt.

```
bool Elso(vector<bool>& R, int n){
  if(R.size()==n) return false;
  R.push_back(1);
  return true;
}

bool Testver(vector<bool>& R, int n){
  if(R.size()==n) return false;
  if(R.size()==0 || R[R.size()-1]==0) return false;
  R[R.size()-1]=0;
  R.push_back(1);
  return true;
}
void rGener(vector<bool> R, int n){
  //R feldolgozása
  for(int i=0;i<R.size();i++)
    if(R[i]) {cout<<i+1<<",";} cout<<endl;
  vector<bool> mR=R;//lokális mentés
  if(Elso(R, n))
    rGener(R, n);
  if(Testver(mR, n))
    rGener(mR, n);
}
```

7.2. Feladat: Pénzváltás

Ki kell fizetnünk adott K összeget adott pénzek felhasználásával úgy, hogy minden pénzt legfeljebb egyszer használhatunk.
Készíts programot, amely kiszámít egy megoldást, ha van!

Bemenet

A standard bemenet első sorában két egész szám van, a kifizetendő K összeg és a rendelkezésre álló pénzek N száma. A második sor N egész számot tartalmaz, a felhasználható pénzek értékeit.

Kimenet

A standard kimenet első és egyetlen sorába azoknak a pénzeknek az értékeit, tetszőleges sorrendbe, amelyek összege pontosan K. Ha nincs megoldás, akkor egy üres sort kell kiírni. Több megoldás esetén bármelyik megadható.

Példa

Bemenet	Kimenet
23 6 9 10 2 7 3 2	9 10 2 2

Korlátok

$1 \leq N \leq 20$
$0 < K < 2\,000\,000\,000$
$0 < p_i \leq 2\,000\,000\,000$
Időlimit: 1.0 másodperc
Memórialimit: 64 MB.

7.3. Megoldás

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int maxN=20;
4  int P[maxN];
5  int K;
6
7  bool Reszhalmazok(int N, vector<int>& R){
8    //Az R részhalmaz feldolgozása
9    int Rk=0;
10   for(int x:R) Rk+=P[x];
11   if(Rk==K) return true;
12
13   int i=R.size()==0 ? 1:R.back()+1;
14   //R[0],...,R[R.size()-1] összes folytatásának előállítása
15   for(int j=i;j<=N;j++){
16     R.push_back(j);
17     if(Reszhalmazok(N, R)) return true;
18     R.pop_back();
19   }
20   return false;
21 }
22
23 int main(){
24   int N;
25   cin>>K>>N;
26   for(int i=1;i<=N;i++)
27   cin>>P[i];
28   vector<int> R;
29   bool van=Reszhalmazok(N, R);
30   if(van) for(int x:R) cout<<P[x]<<" ";
31   cout<<endl;
32 }
```

8. Feladat: Sudoku2: Mester: Visszalépéses keresés

A Sudoku játék 4x4-es változatában egy 4x4-es táblázatot kell kitölteni úgy, hogy minden sorban, minden oszlopban és mind a négy sarok 2x2-es sarokban az 1,2,3,4 számok mindegyike előforduljon!

Készíts programot, amely adott, nem teljesen kitöltött táblázatra kiszámítja, hogy hányféleképpen lehet kitölteni úgy, hogy szabályos sudoku táblázat legyen!

Bemenet

A standard bemenet négy sort tartalmaz, minden sorban négy egész szám van egy-egy szó-közzel elválasztva. A számok értéke 0,1,2,3,4 lehet. A 0 azt jelenti, hogy a táblázat azon elemét meg kell adni a megoldásban.

Kimenet

A standard kimenet első és egyetlen sorába egy egész számot kell írni, ahányféleképpen ki-tölthető a bemeneti táblázat szabályos sudoku táblázattá!

Példa

```
Bemenet   Kimenet
0 3 0 1   2
0 1 0 3
3 0 1 0
0 2 0 4
```

Magyarázat: a két lehetséges elrendezés

```
2 3 4 1   4 3 2 1
4 1 2 3   2 1 4 3
3 4 1 2   3 4 1 2
1 2 3 4   1 2 3 4
```

9. Gyakorló feladatok

https://codeforces.com/problemset/problem/2162/B Beautiful String
https://codeforces.com/problemset/problem/2163/A Souvlaki VS. Kalamak
https://codeforces.com/problemset/problem/2156/A Pizza Time
https://codeforces.com/problemset/problem/2156/B Strange Machine
https://codeforces.com/problemset/problem/2143/A All Lengths Subtraction
https://codeforces.com/problemset/problem/2137/E Mexification